

TP2 — Pipeline data temps réel & plateforme data

Prolongement du TP1 — Immobilier × Énergie, Loire-Atlantique (44)

Louis Valette

Alexis Fouquet

Ruben Cofflard

Contents

1. Architecture	1
2. Sources	2
3. Data lake	2
4. Agrégation et nettoyage	3
5. Observabilité (Prometheus + Grafana)	3
6. Data visualization	3
7. Vérification	4

1. Architecture

Le TP1 a construit une base PostgreSQL statique (import unique, département 44) croisant DVF et DPE. Le TP2 ajoute un pipeline **temps réel** rechargeant la même base en continu : Kafka, data lake brut, Spark Structured Streaming, PostgreSQL, monitoring (Grafana) et dataviz métier (Metabase, outil dédié).

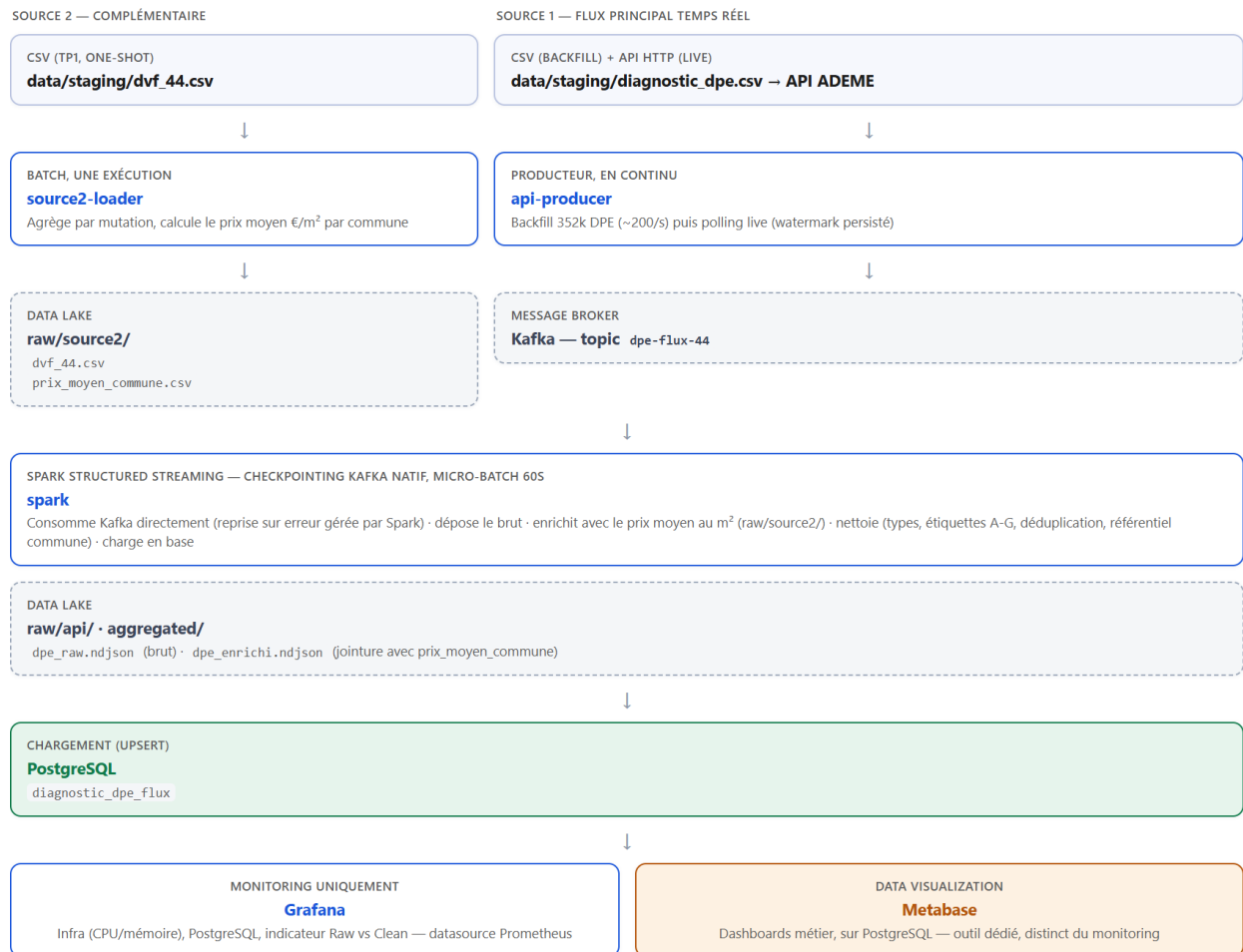


Figure 1: Architecture du pipeline temps réel TP2

2. Sources

#	Source	Nature	Rôle
1	API ADEME dpe03existant	API HTTP, alimentée sur Kafka par api-producer	Flux principal temps réel
2	DVF (département 44, déjà utilisée en TP1)	CSV, chargée en un lot par source2-loader	Enrichit le flux 1 (prix moyen au m² par commune)

L'API ADEME ne produit que quelques nouveaux DPE par semaine pour le seul département 44 — insuffisant pour une démo courte. api-producer rejoue donc d'abord les 352k DPE du TP1 à débit maîtrisé (200/s, source_flux=backfill), puis bascule en interrogation continue de l'API réelle (source_flux=live, watermark persisté). Le backfill ne s'exécute qu'une fois (/state/backfill_done).

3. Data lake

MinIO (S3, proposé par le sujet) a fermé ses images Docker officielles derrière une authentification (Docker Hub et quay.io renvoient 401), incompatible avec un docker compose up reproductible sans compte préalable. Le

data lake est un volume Docker nommé, organisé selon l'arborescence de l'énoncé :

raw/api/	dpe_raw.ndjson	– flux Kafka brut, non filtré
raw/source2/	dvf_44.csv, prix_moyen_commune.csv	
aggregated/	dpe_enrichi.ndjson	– flux enrichi, avant nettoyage

4. Agrégation et nettoyage

spark/jobs/clean_and_load.py consomme le topic Kafka dpe-flux-44 directement en Spark Structured Streaming (micro-batch 60s, checkpointing Kafka natif). Par micro-batch :

1. dépôt du brut dans raw/api/ ;
2. **enrichissement** : jointure avec le prix moyen au m² de la commune (calculé par source2-loader, même correction qu'en TP1 : agrégation par mutation avant calcul du prix/m²) → aggregated/ ;
3. **nettoyage** : types, étiquettes DPE/GES dans A-G, déduplication sur numero_dpe, rejet des communes hors référentiel (FK vers commune) ;
4. upsert dans diagnostic_dpe_flux ([sql/06_create_flux_table.sql](#), ON CONFLICT DO NOTHING).

Le connecteur Kafka de Spark est résolu et figé dans l'image au build, pas téléchargé au démarrage : docker compose up -d ne dépend d'aucun accès réseau à l'exécution.

5. Observabilité (Prometheus + Grafana)

Consigne du formateur : Grafana réservé au monitoring technique ; la dataviz métier passe par un outil séparé (section suivante).

Cible	Exportateur	Expose
Conteneurs	docker-stats-exporter (custom, API Docker)	CPU/mémoire — remplace cAdvisor (échoue sous Docker Desktop Windows/Mac : conteneurs dans une VM cachée, /var/lib/docker monté ne correspond pas au vrai backend) et node-exporter (un seul hôte)
PostgreSQL api-producer, spark	postgres-exporter prometheus_client	Connexions, disponibilité, taille Métriques métier du pipeline

Indicateur Raw vs Clean (obligatoire) : raw_records_total (volume déposé dans raw/api/) vs clean_records_total (COUNT(*) sur diagnostic_dpe_flux), tous deux recalculés depuis le volume persisté (pas un compteur remis à zéro au redémarrage). raw ≥ clean en permanence.

Dashboard TP2 — Pipeline temps réel provisionné automatiquement : 8 panels (Raw vs Clean, taux de nettoyage, débit du producteur, PostgreSQL, CPU/mémoire).

6. Data visualization

Outil dédié, distinct de Grafana (consigne du formateur) : Metabase, connecté à PostgreSQL — répartition des étiquettes DPE, communes les plus chères, part F-G par tranche de prix. Premier accès sur : 3001 : assistant de configuration Metabase, seule étape manuelle du projet (propre à l'outil, qui ne permet pas de provisionner ses dashboards comme du code).

7. Vérification

Stack testée à froid (`docker compose down -v && docker compose up -d`) sur trois machines et réseaux différents, y compris reprise après un `docker kill` de Spark en plein flux (checkpoint Kafka : ni perte ni doublon).

Étape	Résultat
Kafka (dpe-flux-44)	Backfill complet : 351 959 événements publiés
PostgreSQL (diagnostic_dpe_flux)	351 959 lignes après backfill complet
Raw vs Clean	Vérifié à la source (fichiers + base) : raw ≥ clean à tout instant
Prometheus	4/4 cibles UP
Grafana	Dashboard monitoring provisionné ; requêtes vérifiées avec données réelles

Le classement des communes les plus chères, recalculé en continu, retombe sur les valeurs du TP1 (La Baule-Escoublac 6 536 €/m², Pornichet 5 636 €/m², Nantes 3 777 €/m²).

Comment reproduire

```
docker compose up -d
```

Démarre l'ensemble (TP1 + TP2) sans étape manuelle ni accès réseau obligatoire (le backfill utilise les CSV déjà versionnés).

Comment vérifier

Composant	Commande
Kafka	<code>docker exec tp2_kafka /opt/kafka/bin/kafka-get-offsets.sh --bootstrap-server localhost:19092 --topic dpe-flux-44</code>
Data lake	<code>docker exec tp2_spark sh -c "wc -l /datalake/raw/api/dpe_raw.ndjson /datalake/aggregated/dpe_enrichi.ndjson"</code>
PostgreSQL	<code>docker exec tp_audit_44_pg psql -U postgres -d audit_immo_energie_44 -c "SELECT COUNT(*) FROM diagnostic_dpe_flux;"</code>
Prometheus	<code>http://localhost:9090/targets</code>
Grafana	<code>http://localhost:3000 (admin/admin)</code>
Metabase	<code>http://localhost:3001</code>

Ports exposés

Port	Service
5432	PostgreSQL
9092	Kafka (accès externe)
8001 / 8003 / 8004	Métriques Prometheus : producteur / Spark / docker-stats-exporter

Port	Service
9187	postgres-exporter
9090	Prometheus
3000	Grafana
3001	Metabase